个项目工作总结

边缘 AI 云台跟随系统

Jetson Orin Nano + STM32 + 总线舵机 全链路自研

项目信息	
项目周期	2026 年 10 月 17 日（集中开发 1 天）
担任角色	全栈开发（硬件选型 / 嵌入式固件 / 视觉算法 / 系统集成）
技术方向	边缘 AI、嵌入式系统、运动控制
项目性质	个人独立完成（从零设计到交付）
代码仓库	github.com/ZELDA-LINK-F/edge-ai-gimbal

关键词：边缘计算 · 目标检测 · 云台控制 · 嵌入式 · 系统集成

一、项目概述

1.1 项目背景

在边缘 AI 学习中，多数示例停留在「模型能在开发板上跑起来」的阶段。本项目希望更进一步：让 AI 的推理结果直接驱动物理世界，形成「感知 → 决策 → 执行」的完整闭环。

传统方案依赖高性能 PC + 独立显卡，功耗高、体积大、不适合移动场景。本项目选用 Jetson Orin Nano 作为边缘计算平台，验证在 15W 级功耗下实现实时视觉决策与机械执行控制的可行性。

1.2 项目目标

- 在边缘设备上实现实时目标检测（≥30 FPS）
- 将检测结果转换为机械动作，驱动云台跟随目标
- 全链路自研，不依赖现成云台 / 机器人套件
- 形成可复现、可维护、可交付的工程成果

1.3 系统架构

系统采用三层架构：感知层、决策层、执行层。

层级	硬件	软件	职责
感知层	IMX219 CSI 摄像头	camera.py / GStreamer	图像采集，1920×1080@30fps
决策层	Jetson Orin Nano Super	YOLOv8n + TensorRT	目标检测、偏移计算、角度映射
执行层	STM32F103VET6 + LX-16A	裸机固件 + servo.py	命令解析、总线协议、舵机驱动

数据流：摄像头帧 → YOLO 检测 → 人的水平中心 → 归一化偏移 → 舵机目标位置 → 平滑滤波 → 串口命令 → STM32 帧编码 → 舵机转动。端到端延迟 < 50ms。

二、工作内容

2.1 硬件选型与系统接线

- 完成 Jetson、STM32、总线舵机、独立电源的系统级接线设计
- 解决「双电源共地」问题：用面包板汇流电池负极、舵机 GND、板级 GND

- 设计无上拉电阻的信号方案：TX 推挽发送后立即切高阻，RX 内部上拉提供总线电平
- 制定摄像头 FPC 排线保护策略（限位、走线、固定）

2.2 STM32 裸机固件开发（从零）

- 手写启动文件、中断向量表、链接脚本，不依赖 HAL / 标准外设库
- 寄存器级实现 72MHz 系统时钟、USART1/USART2 双串口、SysTick 毫秒计时
- 设计非阻塞行缓冲命令解析，解决高波特率下的丢字符问题
- 实现 LX-16A 二进制帧编码、位置控制、读位置、改 ID、装载/卸载
- 编译产物仅 4.5 KB，无任何第三方依赖

2.3 LX-16A 总线舵机协议实现

- 研读官方协议文档，实现完整帧格式与校验和计算
- 定位并修正帧长度计算错误（多传 2 字节导致舵机静默丢弃）
- 实现双舵机 ID 管理与广播控制
- 处理边界情况：限位过冲导致的负坐标（0xFFFE = -2）

2.4 Jetson 视觉与跟随算法

- 搭建 GStreamer 摄像头管线，通过 90 帧预热解决自动曝光未收敛问题
- 部署 YOLOv8n，导出 TensorRT FP16 引擎，640 输入加速 1.39 倍
- 实现跟随算法：检测框 → 水平偏移 → 角度映射 → 指数平滑滤波
- 设计安全限位（±24°）保护摄像头硬排线

2.5 工程效率工具

- 开发 FlyMcu 自动烧录脚本：INI 预置 + 窗口置顶 + 真实鼠标事件 + 日志校验
- 固件内置诊断命令：LOOP（回环自测）、SCAN（ID 扫描）、BAUD（波特率扫描）
- 建立系统化排查清单，将平均排障时间从小时级降至分钟级

三、技术难点与解决方案

#	难点	分析	解决方案
1	舵机完全无响应	自写协议帧多传 2 字节速度参数，Length 字段错误，舵机静默丢弃整包	逐字节比对官方例程，修正 Length = ID + Cmd + 参数 + 校验和
2	摄像头设备消失	系统批量更新重置 extlinux.conf，DEFAULT 从 JetsonIO 变回 primary，设备树 overlay 未加载	恢复 DEFAULT JetsonIO 并重启，建立更新后配置检查流程
3	舵机总线无信号	PA2/PA3 插在面包板不同横排，两排不导通	内置 LOOP 回环自测命令，5 秒定位线路断点
4	FlyMcu 无法自动化	Delphi 程序不暴露 UI Automation 控件，且窗口被遮挡导致模拟点击落在其他窗口	Win32 枚举子窗口 + 窗口置顶 + 真实鼠标事件，日志文件判定结果
5	FPC 排线易损	硬质排线反复弯折会断裂，传统全角度云台不可行	软件安全限位 ±24° + 走线靠近转轴（扭转代替弯折）+ 根部固定

四、成果与量化数据

4.1 性能指标

指标	数值	说明
检测帧率	37 FPS	YOLOv8n + TensorRT FP16，27ms/帧
检测准确率	100%	连续 3509 帧全部检出目标
舵机定位精度	±1 / 1000	约 ±0.24°，指令与实际位置误差
端到端延迟	< 50 ms	从图像采集到机械动作
固件体积	4.5 KB	裸机、无第三方库
代码量	约 1500 行	C 语言固件 + Python 应用

4.2 跟随实测

测试条件：摄像头固定，人员横向走动，连续采集 2 分钟。

统计项	结果
总处理帧数	3509 帧
成功检出	3509 帧（100%）
目标位置范围	x = 346 ~ 1055（跨越画面 55% 宽度）
舵机跟随范围	454 ~ 563（摆动 109 单位 ≈ 26°）
系统表现	实时跟随，无明显滞后

4.3 交付物

- GitHub 开源仓库：20 个文件，含完整源码与文档
- 技术文档 8 篇（约 42 KB）：架构、接线、固件、协议、视觉、算法、踩坑、性能
- 可运行系统：STM32 固件 + Jetson 应用 + 自动烧录工具

五、学习收获与反思

5.1 技术能力提升

方向	具体收获
嵌入式开发	掌握寄存器级编程、启动流程、中断向量表、链接脚本、时钟树配置
协议实现	理解二进制协议的严谨性，掌握帧格式设计、校验、边界处理
边缘 AI	掌握模型部署、TensorRT 加速、性能瓶颈定位（预处理 vs 推理）
系统集成	跨平台（Windows / Linux）、跨语言（C / Python）联合调试能力
工程化	自动化脚本、诊断工具、可复现构建流程

5.2 工程方法反思

- 协议文档必须逐字节核对，写完先用官方例子做比对 —— 本次最大教训
- 自制工具在同一环节反复失败时，应先用成熟工具验证硬件，避免在错误方向深挖
- GUI 自动化失败时，先截图确认实际屏幕状态，不要凭空推测
- 硬件调试需要系统化排查清单（本项目已沉淀为文档）
- 自动化投入产出比很高：一条命令烧录，当天迭代十余次固件

5.3 不足与改进

- 协议实现前应先完整阅读文档，而非凭记忆编写
- 机械结构尚未完成最终固定，当前为逻辑验证阶段
- 测试覆盖不足：缺少自动化回归测试

六、后续规划

阶段	内容	预期产出
近期	摄像头装至舵机、机械结构完善	真正的云台跟随（摄像头随舵机转动）
近期	增加俯仰舵机（ID2）	二维云台，支持上下注视

中期	接入视觉大模型（VLM）按需识别	可回答「画面里是什么」
中期	接入语音控制	语音指令控制云台
长期	人脸跟随、目标锁定	更精细的追踪能力
长期	性能优化与功耗测量	完整的边缘设备性能报告

— 报告完 —